Guía N° 1: Boot de Micro S.O. en QEMU

Informe integrador – Fases 1, 2 y 3

INFB6074 – Infraestructura para Ciencia de Datos – UTEM – 2026-2

1 Propósito y entorno

Esta guía propone construir, en tres etapas progresivas, un flujo completo de arranque de bajo nivel: desde un sector BIOS mínimo que dibuja una imagen en modo texto, pasando por un arranque en dos etapas con interacción por teclado, hasta un kernel en C que corre en modo protegido de 32 bits y dibuja una imagen propia en un framebuffer gráfico. El propósito formativo es comprender, de manera práctica, cómo se articulan código fuente, ensamblado, artefacto binario, arranque y virtualización – la base conceptual sobre la que luego se apoyan máquinas virtuales, contenedores e imágenes ejecutables.

Entorno de desarrollo: Windows 11 x64. Ensamblador NASM 3.02. Emulador QEMU 11.1.2 (`qemu-system-i386`). Compilador `mingw-w64-ucrt-x86_64-gcc` 16.2.0 (con soporte `-m32`), usado únicamente en la Fase 3. Todas las herramientas se obtuvieron del repositorio oficial de MSYS2 (`pacman`). Control de versiones con Git; repositorio personal alojado en GitHub.

2 Fase 1: sector de arranque e imagen ASCII

Se implementó un sector de arranque BIOS de 512 bytes (`fase-1/boot_f1.asm`) que usa la interrupción `int 0x10` (función `AH=0x0E`, modo teletipo) para dibujar, en modo texto 80×25, una adaptación del escudo UTEM hecha con caracteres ASCII. El flujo de construcción es `nasm -f bin boot_f1.asm -o boot_f1.bin`, seguido de QEMU apuntando ese binario como disquete en bruto (`-drive format=raw,if=floppy`). Se verificó que el binario mide exactamente 512 bytes y que sus dos últimos bytes son la firma de arranque `55 AA`, requisito que la BIOS exige para reconocer el sector como arrancable. El arranque se probó visualmente en QEMU: el dibujo completo aparece en pantalla y el sistema queda detenido (`cli; hlt`) de forma controlada.

3 Fase 2: arranque en dos etapas e interacción

La Fase 2 amplía el flujo anterior en dos binarios: un sector de arranque de 512 bytes (`boot_f2.asm`) que usa `int 0x13` con lectura CHS para cargar una segunda etapa de 2048 bytes (`stage2_f2.asm`) desde el sector 2 del disco hacia la dirección física `0x8000`, y le transfiere el control con un salto lejano. La segunda etapa conserva el logo ASCII y agrega un menú de tres opciones leído con `int 0x16` (`AH=0x00`): dos opciones informativas (NASM y QEMU) y una para detener el sistema; cualquier otra tecla produce un mensaje de entrada no válida y vuelve a mostrar el menú, sin bloquear el sistema.

Problema encontrado y corregido. La primera versión reutilizó, para los mensajes del menú, la misma rutina que dibuja el logo – una rutina que reposiciona manualmente el cursor en cada salto de línea. Como los mensajes normales ya incluían retorno de carro y salto de línea (`13,10`) para que la BIOS los maneje en modo teletipo, esa reposición manual pisaba el cursor y cortaba el primer carácter de cada línea (por ejemplo, *"Seleccione"* aparecía como *"eleccione"*). La corrección separó dos rutinas: una con control manual de cursor, sólo para el dibujo, y otra en modo teletipo puro para el resto del texto. Se verificó con capturas automatizadas (enviando teclas y volcando pantalla a través del monitor de QEMU) que los cinco casos de prueba – arranque, opción 1, opción 2, entrada no válida y detención – funcionan correctamente.

4 Fase 3: cargador propio, kernel C y framebuffer gráfico

La Fase 3 reutiliza la cadena de arranque propia en lugar de GRUB + Multiboot2, ya que este equipo no tiene instalados `grub-mkrescue` ni `xorriso`. El sector de arranque (`boot_f3.asm`) y un cargador en modo real de 2048 bytes (`loader_f3.asm`) cargan, con lectura extendida por LBA (`int 0x13`, `AH=0x42`), tanto el propio cargador como el kernel compilado (de aproximadamente 28 KB), superando el límite de 18 sectores por pista de la lectura CHS clásica.

Antes de activar el modo gráfico, el cargador consulta la información del modo VBE (`int 0x10`, `AX=0x4F01`) y la guarda en la dirección física fija `0x9000`; recién después activa el modo `0x112` (640×480, con buffer lineal). Luego habilita A20, construye una tabla de descriptores global plana de 4 GB y conmuta a modo protegido de 32 bits, saltando finalmente a la dirección física `0x20000`, donde corre el kernel en C.

El kernel (`kernel.c`) interpreta el bloque de información VBE con una estructura que replica sus offsets oficiales: dirección física del framebuffer, bytes por línea, resolución, bits por píxel y posición de cada canal de color. **Hallazgo relevante:** se asumió inicialmente que el modo `0x112` entregaría 32 bits por píxel; al inspeccionar en vivo el bloque real (con el monitor de QEMU) se confirmó que en este equipo entrega 24 bits por píxel. La primera versión del kernel validaba `bpp == 32` y, al no cumplirse, omitía todo el dibujo sin mostrar error (pantalla negra). La corrección dejó de asumir un formato fijo: el kernel compone cada píxel usando las posiciones de color que realmente informa la BIOS y escribe 3 o 4 bytes según corresponda – la validación de parámetros que exige la guía antes de escribir en memoria no es un formalismo: sin ella, el kernel escribía con un formato incorrecto.

El kernel dibuja primero un color sólido uniforme (azul institucional UTEM) para confirmar que el framebuffer responde, y luego superpone una imagen propia de 96×64 píxeles (archivo `logo_f3.png` en `assets/original`), convertida a píxeles RGB sin compresión mediante un script documentado (`convertir_imagen.py` en `scripts`) y embebida como arreglo de C. El resultado se verificó visualmente en QEMU: fondo azul uniforme con el emblema centrado, colores y proporciones correctos.

5 Pruebas, problemas, limitaciones, reflexión y referencias

Pruebas. Cada fase incluye su propia tabla de pruebas con caso, acción, resultado esperado, resultado observado y evidencia (ver los `README.md` de `fase-1`, `fase-2` y `fase-3`). En las fases 2 y 3 las pruebas se automatizaron enviando comandos al monitor de QEMU (`sendkey`, `screendump`), lo que permite reproducir exactamente la misma secuencia de verificación.

Problemas resueltos. (1) En la Fase 2, una rutina de impresión mal reutilizada cortaba texto del menú. (2) En la Fase 3, la lectura CHS clásica no alcanza para un kernel de varias decenas de sectores – se migró a lectura extendida por LBA, y se descubrió que la BIOS de disquete emulada por QEMU no la soporta, por lo que la imagen final se monta como disco IDE. (3) También en la Fase 3, una suposición incorrecta sobre el formato de píxel (32 en vez de 24 bpp) dejaba la pantalla en negro sin aviso; se corrigió validando y usando el formato real informado por la BIOS.

Limitaciones. El kernel de la Fase 3 sólo soporta framebuffers de 24 o 32 bpp, no implementa teclado ni interrupciones, y su tamaño máximo está acotado por una constante documentada en el cargador (con verificación explícita en el script de build, sin truncar silenciosamente). Ninguna de las tres fases implementa un sistema de archivos ni gestión de memoria más allá de lo estrictamente necesario para arrancar y dibujar.

Reflexión. Las tres fases hacen tangible la relación entre hardware, firmware (BIOS), virtualización y software: cada capa impone restricciones concretas (tamaño de sector, límites de interrupciones BIOS, formatos de color reales en vez de asumidos) que sólo se descubren ejecutando y depurando en un entorno emulado y controlado, no sólo leyendo documentación. Esa disciplina de

validar en tiempo de ejecución – en vez de asumir – es exactamente la misma que sostiene la reproducibilidad en infraestructura de datos a mayor escala: máquinas virtuales, contenedores e imágenes ejecutables.

Referencias.

- QEMU Project. `https://www.qemu.org/` y `https://www.qemu.org/docs/master/`
- NASM Documentation. `https://www.nasm.us/doc/`
- OSDev Wiki (Bootloader, BIOS, VESA/VBE). `https://wiki.osdev.org/`
- GitHub Docs. `https://docs.github.com/`